

CSA3006 - DATA MINING AND DATA WAREHOUSING

Presented By

Dr Jasmine Selvakumari Jeya I

Senior Associate Professor Grade 2/SCOPE

VIT Bhopal University

Module III

- **Classification:** Basic concepts–Decision tree induction–Bayesian classification, Rule–based classification, Support Vector Machines, Improve Classification Accuracy, Model Evaluation and Selection - Techniques to Improve Classification Accuracy.
- **Regression Analysis:** Linear and Logistic Regression, Neural Networks, Regression Evaluation.

Classification – Basic Concepts

- **Classification** is a supervised learning technique used to assign data objects to predefined class labels based on their attributes.
- **Supervised learning** is a type of machine learning in which the model is trained using a **labeled dataset**.
- Supervised means the learning process is guided by known correct answers.

Example 1: Email Spam Detection

Email Content	Label
"Win money now"	Spam
"Meeting at 10 AM"	Not Spam

- Input → Email text
- Output → Spam / Not Spam
- The model learns from labeled emails and predicts future emails.

Two Main Types of Supervised Learning

Classification

- Output is categorical
- Example: Yes/No, Fraud/Not Fraud

Regression

- Output is continuous
- Example: Salary prediction, Temperature prediction

Common Supervised Algorithms

- Decision Tree
- Naïve Bayes
- Support Vector Machine
- k-Nearest Neighbor
- Logistic Regression
- Neural Networks

Advantages

- High accuracy when labeled data is sufficient
- Easy to evaluate performance
- Clear objective (predict known output)

Disadvantages

- Requires labeled data (expensive/time-consuming)
- May overfit training data

Unsupervised Learning

- Unsupervised learning is a machine learning approach where the model learns from **unlabeled data**.
- There is **no target (output) label** given.
- The algorithm discovers **hidden patterns or groupings** in the data.

- Example 1: Customer Segmentation (Clustering)
- A supermarket wants to group customers based on their purchasing behavior.

Customer ID	Annual Income	Spending Score
C1	25,000	30
C2	30,000	35
C3	80,000	85
C4	85,000	90
C5	40,000	50

- There is **no label** like “High Value Customer” or “Low Value Customer”.

- Using **K-Means Clustering**, it automatically groups customers into clusters:
- Cluster 1 → Low income, low spending
- Cluster 2 → High income, high spending
- Cluster 3 → Medium customers
- ✓ No predefined class labels
- ✓ Patterns discovered automatically

- Example 2: Market Basket Analysis (Association Rules)

Transaction	Items Purchased
T1	Milk, Bread
T2	Milk, Diapers
T3	Bread, Butter
T4	Milk, Bread, Butter

- Using **Apriori algorithm**, the system finds:
- If a customer buys **Milk**, they often buy **Bread**.
- There is no label — only pattern discovery.

- **Example 3: Dimensionality Reduction**

- In image recognition, thousands of pixel features are reduced using:
- Principal Component Analysis (PCA)
- The system reduces features without labeled output.

Common Unsupervised Learning Algorithms

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN
- Apriori Algorithm
- PCA (Principal Component Analysis)

Decision tree induction

- A **Decision Tree** is a supervised machine learning algorithm used for **classification and regression**. It splits the dataset into subsets based on feature values using measures like:
 - **Entropy**
 - **Information Gain**
 - **Gini Index**

1. Entropy

- Entropy measures impurity in data.

$$Entropy(S) = - \sum p_i \log_2 p_i$$

Where:

- p_i = probability of class i

Entropy range:

- 0 → Pure (all same class)
- 1 → Maximum impurity (binary case)

2. Information Gain (IG)

$$IG(S, A) = Entropy(S) - \sum \frac{|S_v|}{|S|} Entropy(S_v)$$

Choose the attribute with **highest Information Gain**.

Steps in Decision Tree Induction

- Calculate Entropy of dataset
- Calculate Information Gain for each attribute
- Choose attribute with highest IG (Root node)
- Split dataset
- Repeat until:
 - All records belong to same class
 - No attributes left

Numerical Problem

- Construct a decision tree using **Information Gain**.

ID	Weather	Temperature	Play
1	Sunny	Hot	No
2	Sunny	Mild	No
3	Overcast	Hot	Yes
4	Rain	Mild	Yes
5	Rain	Cool	Yes
6	Sunny	Cool	Yes

- Target: **Play (Yes/No)**

Step 1: Calculate Entropy of Dataset

- Total records = 6
Yes = 4
No = 2

$$P(Yes) = 4/6 = 0.667$$

$$P(No) = 2/6 = 0.333$$

$$\begin{aligned} Entropy(S) &= -(0.667 \log_2 0.667 + 0.333 \log_2 0.333) \\ &= -(0.667 \times -0.585 + 0.333 \times -1.585) \\ &= 0.918 \end{aligned}$$

ID	Weather	Temperature	Play
1	Sunny	Hot	No
2	Sunny	Mild	No
3	Overcast	Hot	Yes
4	Rain	Mild	Yes
5	Rain	Cool	Yes
6	Sunny	Cool	Yes

$$Entropy(S) = - \sum p_i \log_2 p_i$$

$$\log_2 x = \frac{\log x}{\log 2}$$

Step 2: Information Gain for Weather

- **Weather = Sunny (3 records)**

Yes = 1

No = 2

$$\begin{aligned} \text{Entropy}(\text{Sunny}) &= -(1/3 \log_2 1/3 + 2/3 \log_2 2/3) \\ &= 0.918 \end{aligned}$$

ID	Weather	Temperature	Play
1	Sunny	Hot	No
2	Sunny	Mild	No
3	Overcast	Hot	Yes
4	Rain	Mild	Yes
5	Rain	Cool	Yes
6	Sunny	Cool	Yes

$$\text{Entropy}(S) = - \sum p_i \log_2 p_i$$

Weather = Overcast (1 record)

Yes = 1

Entropy = 0 (Pure)

Weather = Rain (2 records)

Yes = 2

Entropy = 0 (Pure)

Weighted Entropy: $= (3/6 \times 0.918) + (1/6 \times 0) + (2/6 \times 0)$
 $= 0.459$

Information Gain (Weather)

• $IG(\text{Weather}) = 0.918 - 0.459$

IG=0.459

$$Entropy(S) = - \sum p_i \log_2 p_i$$

Step 3: Information Gain for Temperature

Hot (2 records)

Yes = 1

No = 1

Entropy = 1

Mild (2 records)

Yes = 1

No = 1

Entropy = 1

Cool (2 records)

- Yes = 2

- No=0

Entropy = 0

ID	Weather	Temperature	Play
1	Sunny	Hot	No
2	Sunny	Mild	No
3	Overcast	Hot	Yes
4	Rain	Mild	Yes
5	Rain	Cool	Yes
6	Sunny	Cool	Yes

Weighted Entropy

$$\begin{aligned} &= (2/6 \times 1) + (2/6 \times 1) + (2/6 \times 0) \\ &= 0.667 \end{aligned}$$

Information Gain (Temperature)

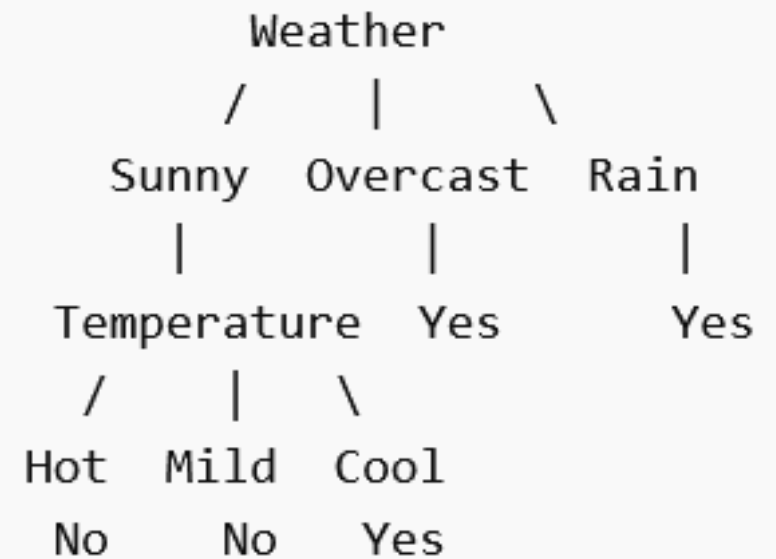
$$\begin{aligned} IG(Temperature) &= 0.918 - 0.667 \\ &= 0.251 \end{aligned}$$

Step 4: Choose Root Node

Attribute	Information Gain
Weather	0.459
Temperature	0.251

Weather has highest IG → Root Node

Final Decision Tree



Bayesian Classification

- Bayesian Classification is a **probabilistic supervised learning method** based on **Bayes' Theorem**.
- It predicts the class label based on **probability values**.
- Most common model: **Naïve Bayes Classifier**
- Bayes' Theorem

$$P(C|X) = \frac{P(X|C) P(C)}{P(X)}$$

Term	Meaning
C	Class (Yes/No, Spam/Not Spam)
X	Input features (attributes)
P(C)	Prior Probability of class
P(X)	Evidence

- **Prior ($P(C)$)** → Probability of class before seeing data
- **Likelihood ($P(X|C)$)** → Probability of data given class
- **Posterior ($P(C|X)$)** → Final probability after considering data

- **Naïve Bayes Assumption**
- Naïve Bayes assumes:
- All attributes are **independent**
- So:

$$P(X|C) = P(x_1|C) \times P(x_2|C) \times \dots \times P(x_n|C)$$

- Given the following dataset:

ID	Fever	Cough	Flu
1	Yes	Yes	Yes
2	Yes	No	Yes
3	No	Yes	No
4	Yes	Yes	Yes
5	No	Yes	No
6	No	No	No

- Predict class for:
- **Fever = Yes, Cough = Yes**

- **Step 1: Calculate Prior Probabilities**

- Total records = 6

- Flu = Yes → 3

- Flu = No → 3

$$P(Yes) = 3/6 = 0.5$$

$$P(No) = 3/6 = 0.5$$

ID	Fever	Cough	Flu
1	Yes	Yes	Yes
2	Yes	No	Yes
3	No	Yes	No
4	Yes	Yes	Yes
5	No	Yes	No
6	No	No	No

- **Step 2: Likelihood Calculation**

- **For Flu = Yes**

- Rows with Flu = Yes:

Fever	Cough
Yes	Yes
Yes	No
Yes	Yes

ID	Fever	Cough	Flu
1	Yes	Yes	Yes
2	Yes	No	Yes
3	No	Yes	No
4	Yes	Yes	Yes
5	No	Yes	No
6	No	No	No

- Count = 3

$$P(Fever = Yes | Flu = Yes) = 3/3 = 1$$

$$P(Cough = Yes | Flu = Yes) = 2/3$$

- **For Flu = No**

- Rows with Flu = No:

- Count = 3

Fever		Cough	3	No	Yes	No
No		Yes	4	Yes	Yes	Yes
No		Yes	5	No	Yes	No
No		No	6	No	No	No

$$P(\text{Fever} = \text{Yes} | \text{Flu} = \text{No}) = 0/3 = 0$$

$$P(\text{Cough} = \text{Yes} | \text{Flu} = \text{No}) = 2/3$$

- Step 3: Apply Naïve Bayes Formula

$$\begin{aligned}P(Yes|X) &\propto P(Fever|Yes) \times P(Cough|Yes) \times P(Yes) \\&= 1 \times (2/3) \times 0.5 \\&= 0.333\end{aligned}$$

$$\begin{aligned}P(No|X) &\propto 0 \times (2/3) \times 0.5 \\&= 0\end{aligned}$$

- Step 4: Final Classification

$$0.333 > 0$$

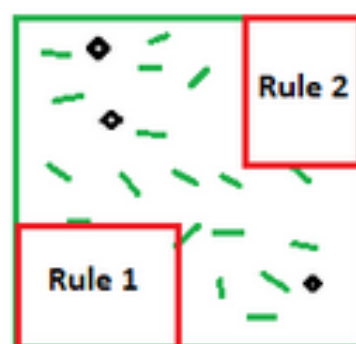
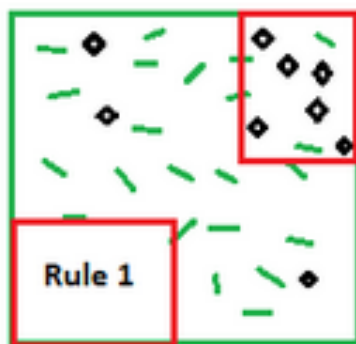
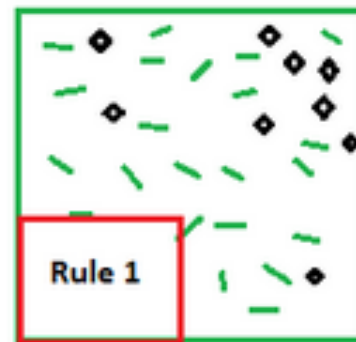
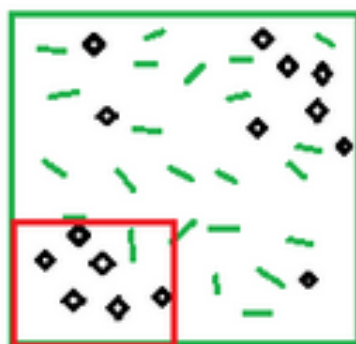
- **Predicted Class = Flu = Yes**

Rule-Based Classification

1. Introduction

- Rule-Based Classification is a supervised learning technique where the model is represented as a set of **IF-THEN rules** used to classify data instances.
- Each rule has:
- **Antecedent (Condition part)** → IF condition
- **Consequent (Class part)** → THEN class label
- **Example:**

```
IF Age > 30 AND Income = High  
THEN Buy_Insurance = Yes
```



— Negative Class

◆ Positive Class

- This diagram illustrates how **Rule-Based Classification** works using the **Sequential Covering (Separate-and-Conquer)** algorithm.

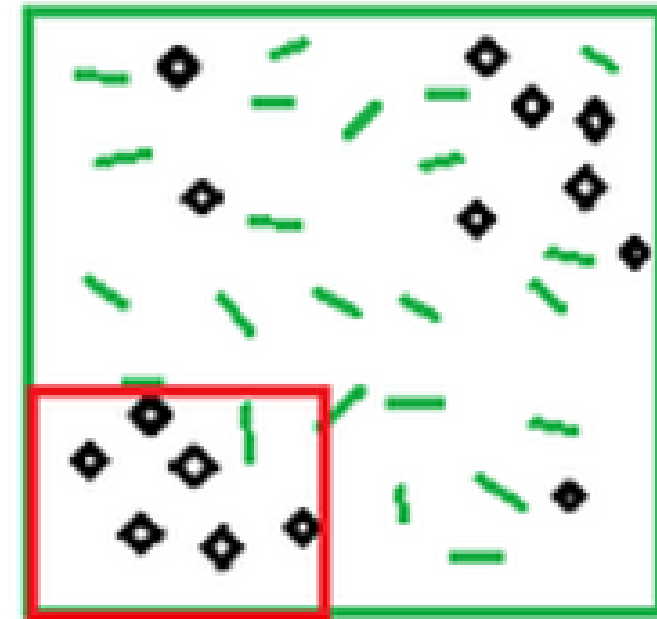
Symbols in the Diagram

- **Green dashes (—)** → Negative Class
- **Black diamonds (◇)** → Positive Class
- **Red rectangle** → Region covered by a rule

Step 1: Identify a Pure Region (Top-Left Image)

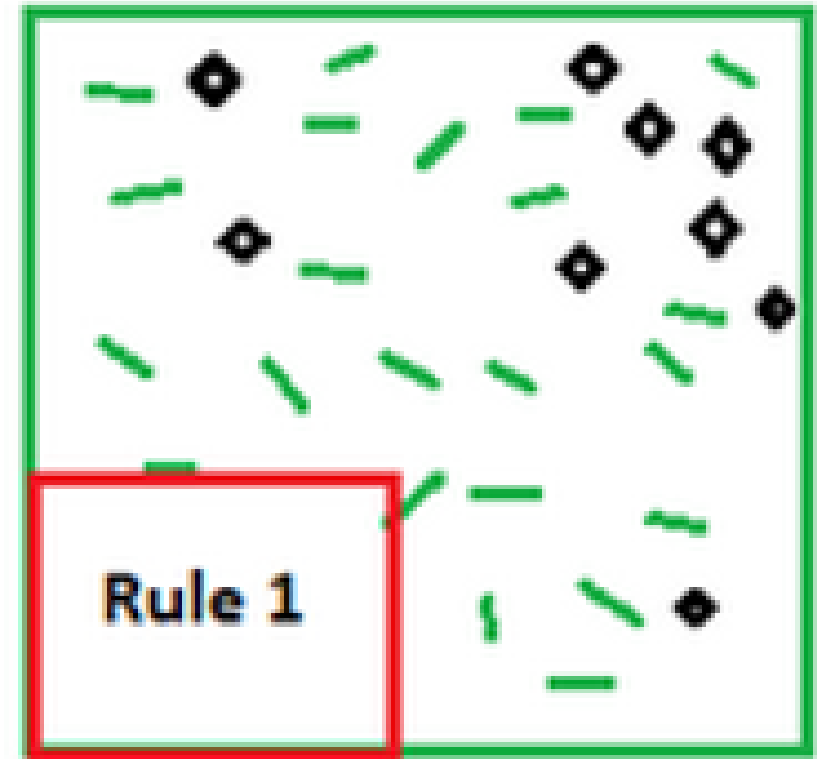
- The algorithm looks for a region containing mostly **positive class** ($\diamond$).
- A red rectangle is drawn around the bottom-left area where many positive samples exist.
- This region forms **Rule 1**.

```
IF (X and Y fall inside this region)  
THEN Class = Positive
```



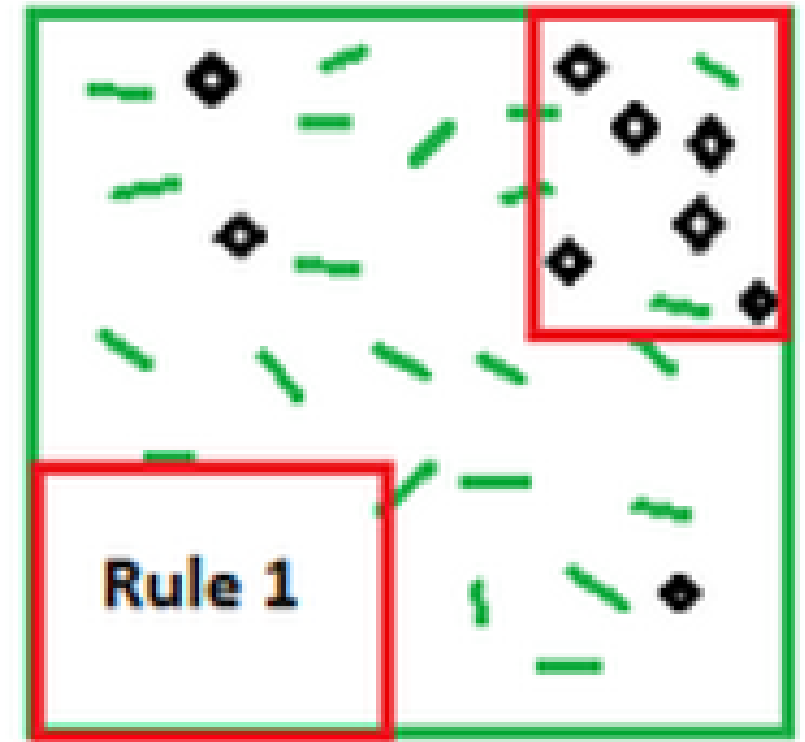
Step 2: Form Rule 1 (Top-Right Image)

- The selected region is labeled as **Rule 1**.
- All positive instances inside that red region are considered **covered**.
- These covered instances will be removed from further consideration.



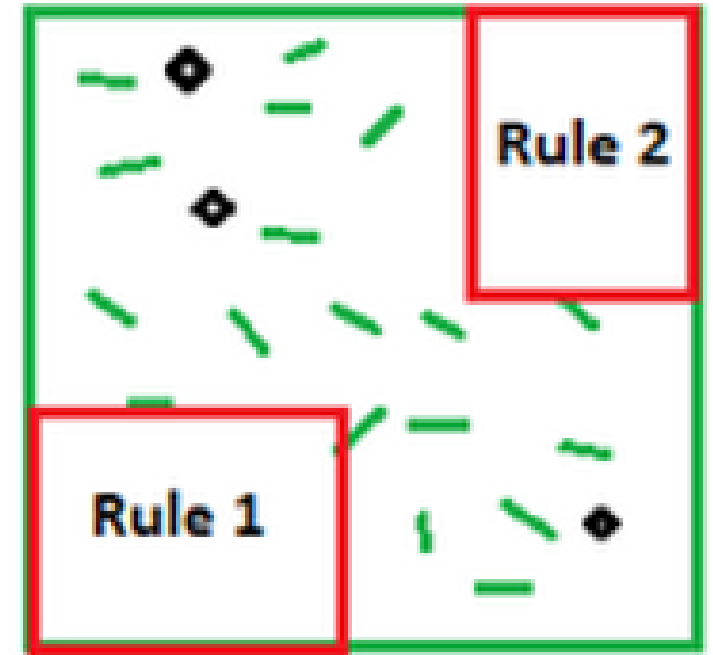
Step 3: Remove Covered Instances & Search Again (Bottom-Left Image)

- After removing instances covered by Rule 1, the algorithm searches for another region containing mostly positive samples.
- A new red rectangle is found in the top-right area.
- This becomes **Rule 2**.



Step 4: Final Rule Set (Bottom-Right Image)

- Now we have:
- **Rule 1** → Covers first cluster of positive samples
- **Rule 2** → Covers second cluster of positive samples
- Remaining instances (if any) are classified using a **default rule** (usually Negative).



2. Structure of a Rule

- A classification rule is generally written as:

IF Condition → Class

- Where:
- Condition = Conjunction of attribute tests
- Class = Predicted class label
- **General Form:**

```
IF (A1 op v1) AND (A2 op v2) AND ...  
THEN Class = C
```

3. Characteristics

- Easy to understand and interpret
- Human-readable model
- Suitable for decision-making systems
- Can handle categorical and numeric data

4. Rule Quality Measures

- To evaluate rules, the following metrics are used:

1. Coverage

- Number of instances satisfying the rule condition.

$$Coverage(R) = \frac{\text{Number of tuples covered by R}}{\text{Total tuples}}$$

2. Accuracy (Confidence)

- Percentage of correctly classified instances.

$$Accuracy(R) = \frac{\text{Correctly classified tuples}}{\text{Tuples covered by R}}$$

5. Types of Rule–Based Classifiers

1. Direct Method

- Rules are extracted directly from training data.
- **Examples:**
- RIPPER
- CN2 Algorithm

2. Indirect Method

- Rules are derived from other models like decision trees.
- **Example:**
- Convert Decision Tree to IF–THEN rules

6. Rule Generation Methods

A. Sequential Covering Algorithm (Separate and Conquer)

Steps:

1. Learn one rule.
2. Remove covered instances.
3. Repeat until stopping condition.
4. This approach is also called:
5. Covering algorithm
6. Separate-and-conquer strategy

7. Ordered vs Unordered Rule Set

Ordered Rule Set (Decision List)

- Rules are evaluated in order.
- First matching rule is applied.
- Has default rule at the end.

Unordered Rule Set

- All matching rules are considered.
- Majority voting or best rule selected.

8. Advantages

- Simple and interpretable
- Fast classification
- Easy to convert into business rules
- Works well with structured data

9. Disadvantages

- May overfit training data
- Difficult to handle noisy data
- Large number of rules may reduce clarity

10. Applications

- Medical diagnosis
- Fraud detection
- Credit approval systems
- Expert systems
- Business decision support systems

Example: Consider the following training dataset:

Tuple	Outlook	Temperature	Humidity	Wind	PlayTennis
T1	Sunny	Hot	High	Weak	No
T2	Sunny	Hot	High	Strong	No
T3	Overcast	Hot	High	Weak	Yes
T4	Rain	Mild	High	Weak	Yes
T5	Rain	Cool	Normal	Weak	Yes
T6	Rain	Cool	Normal	Strong	No
T7	Overcast	Cool	Normal	Strong	Yes
T8	Sunny	Mild	High	Weak	No
T9	Sunny	Cool	Normal	Weak	Yes
T10	Rain	Mild	Normal	Weak	Yes

- Positive Class = **Yes**
Negative Class = **No**
- Using Sequential Covering, generate rules for class **Yes**.
Also compute **Coverage** and **Accuracy** of each rule.

- **Step 1: Generate Rule 1**
- Look for attribute conditions that strongly predict **Yes**.
- Observe:
- All tuples with **Outlook = Overcast** → Yes (T3, T7)
- **Rule 1:**

```
IF Outlook = Overcast
THEN PlayTennis = Yes
```

Tuple	Outlook	Temperature	Humidity	Wind	PlayTennis
T1	Sunny	Hot	High	Weak	No
T2	Sunny	Hot	High	Strong	No
T3	Overcast	Hot	High	Weak	Yes
T4	Rain	Mild	High	Weak	Yes
T5	Rain	Cool	Normal	Weak	Yes
T6	Rain	Cool	Normal	Strong	No
T7	Overcast	Cool	Normal	Strong	Yes
T8	Sunny	Mild	High	Weak	No
T9	Sunny	Cool	Normal	Weak	Yes
T10	Rain	Mild	Normal	Weak	Yes

- **Coverage of Rule 1:**

- Number of tuples covered = 2 (T3, T7)

$$\textit{Coverage} = 2/10 = 0.2$$

- **Accuracy:**

- Correctly classified = 2
Total covered = 2

$$\textit{Accuracy} = 2/2 = 100\%$$

- Remove T3 and T7.

- **Step 2: Generate Rule 2**
- Remaining Yes tuples:
T4, T5, T9, T10
- Observe:
- Most Rain tuples with Wind = Weak → Yes
(T4, T5, T10)
- **Rule 2:**

```
IF Outlook = Rain AND Wind = Weak  
THEN PlayTennis = Yes
```

- **Coverage:**

- Covers T4, T5, T10 → 3 tuples

$$\text{Coverage} = 3/10 = 0.3$$

- **Accuracy:**

- Correct = 3

Total covered = 3

$$\text{Accuracy} = 3/3 = 100\%$$

- Remove T4, T5, T10.

- **Step 3: Generate Rule 3**
- Remaining Yes tuple:
T9
- T9: Sunny, Cool, Normal, Weak → Yes
- Check condition:
- Sunny AND Humidity = Normal → Yes
(Only T9 in remaining data)
- **Rule 3:**

```
IF Outlook = Sunny AND Humidity = Normal  
THEN PlayTennis = Yes
```

- **Coverage:**
- $1/10 = 0.1$
- **Accuracy:**
- $1/1 = 100\%$

- **Default Rule**
- All remaining tuples → No

```
ELSE PlayTennis = No
```

Final Rule Set (Ordered)

- IF Outlook = Overcast → Yes
- IF Outlook = Rain AND Wind = Weak → Yes
- IF Outlook = Sunny AND Humidity = Normal → Yes
- ELSE → No

Improve Classification Accuracy

- Improving **classification accuracy** means increasing the percentage of ***correctly predicted class labels in a machine learning*** or deep learning model.
- This can be achieved by improving **data quality, model design, training strategy, and evaluation methods.**

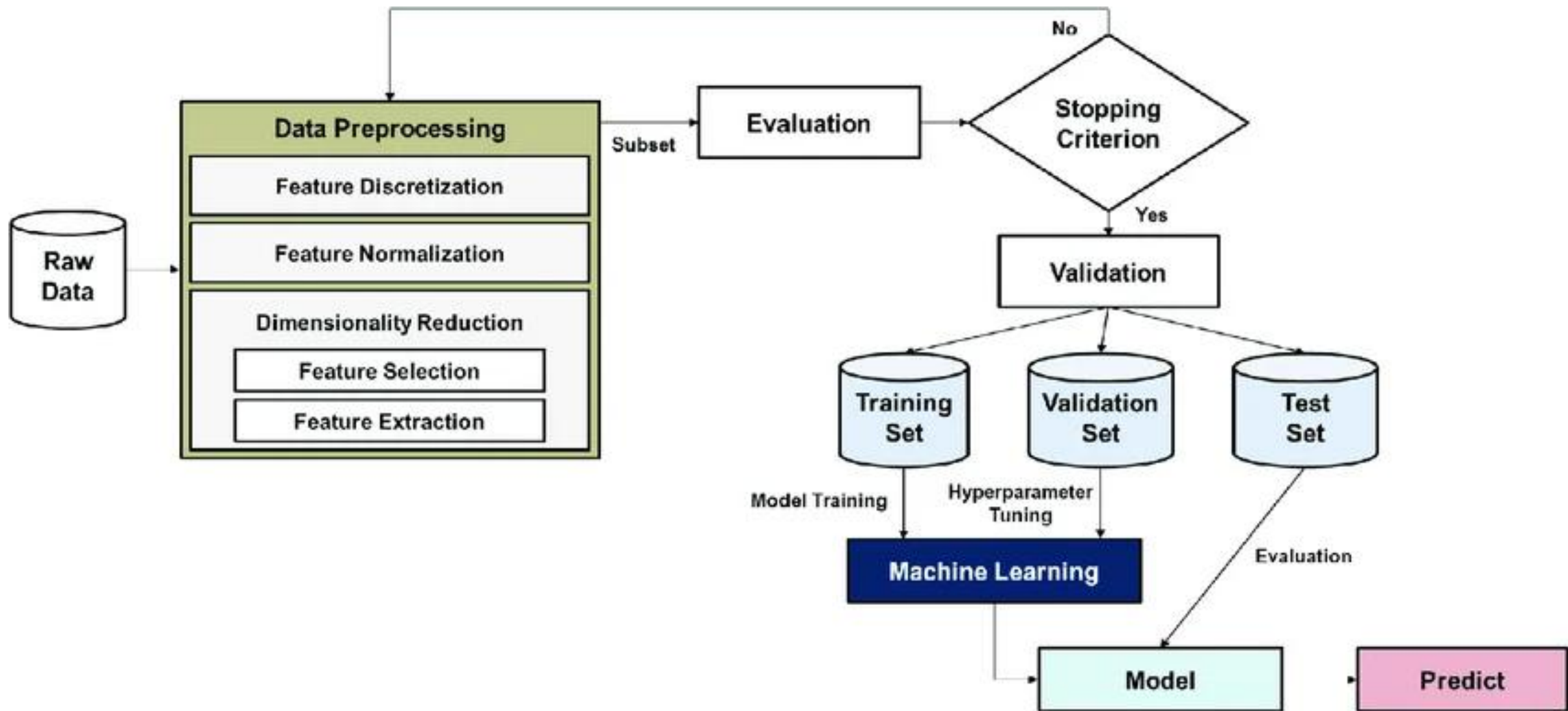
- 1. Data Pre-processing & Feature Engineering
- 2. Handling Imbalanced Dataset
- 3. Model Selection & Hyperparameter Tuning
- 4. Reducing Overfitting
- 5. Ensemble Methods
- 6. Evaluation Metrics (Beyond Accuracy)

1. Data Pre-processing & Feature Engineering

Key Techniques

- **Data Cleaning:** Remove noise, duplicates, and incorrect labels.
- **Handling Missing Values:** Use mean/median/mode imputation or model-based filling.
- **Feature Scaling:**
 - **Normalization:** Scale values between 0 and 1
 - **Standardization:** Transform to zero mean and unit variance
- **Feature Selection:** Remove irrelevant or redundant features.
- **Feature Extraction:** Use PCA, Linear Discriminant Analysis (LDA), or embeddings.

Better input features → Better class separation → Higher accuracy.



Feature scaling

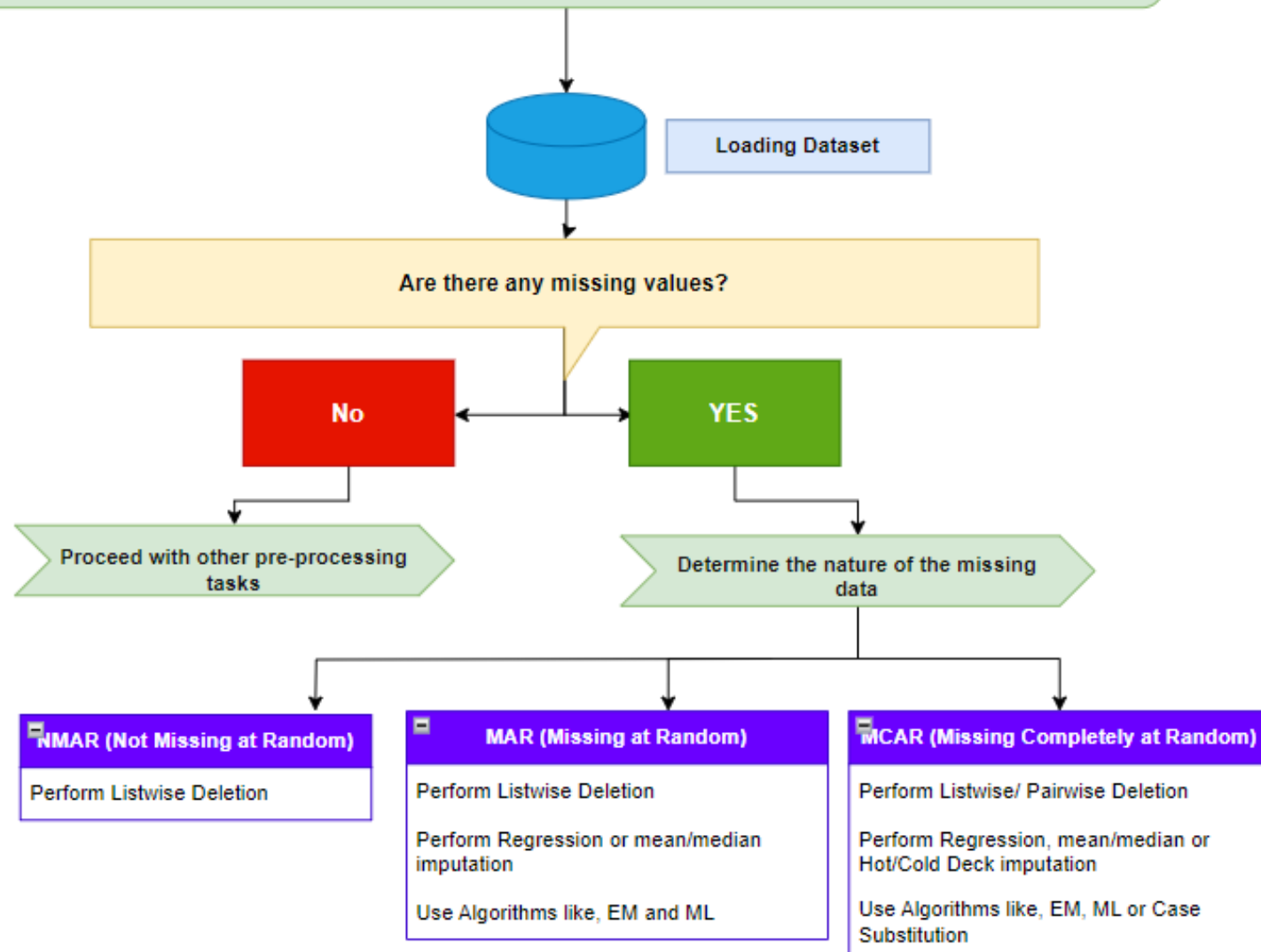
Normalization

Standardization

$$X_{new} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

$$X' = \frac{X - \text{Mean}}{\text{Standard deviation}}$$

PROCESS OF HANDLING MISSING DATA

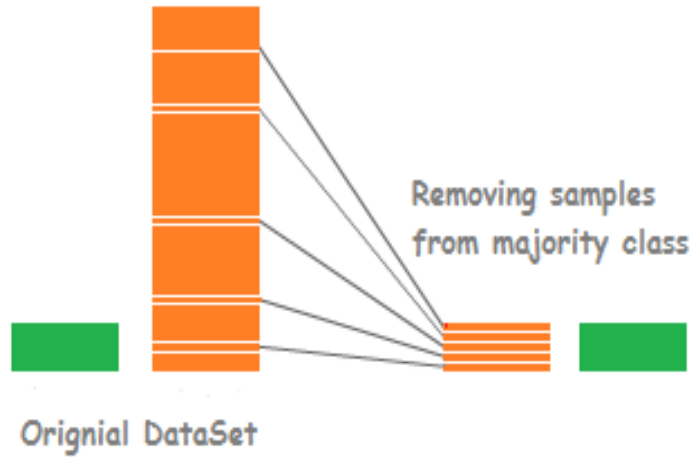


2. Handling Imbalanced Dataset

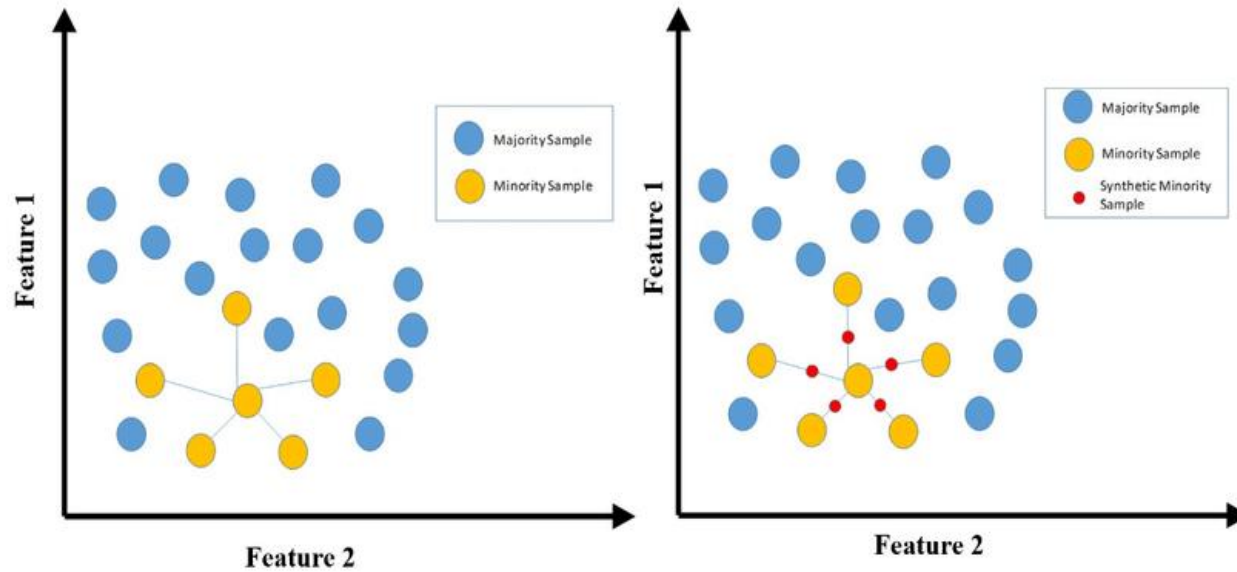
Methods

- **Oversampling:** Duplicate minority class samples.
- **Undersampling:** Reduce majority class samples.
- **Class Weights:** Penalize misclassification of minority class more.
- Prevents model bias toward dominant class.

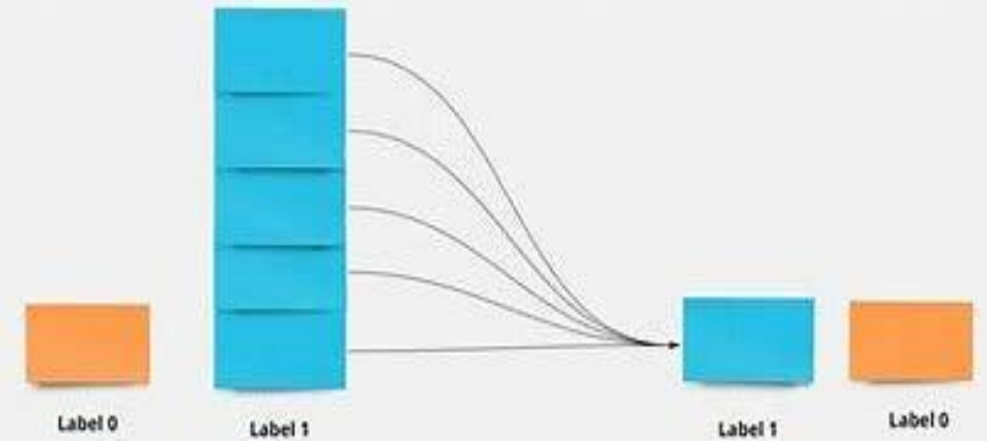
Undersampling



Oversampling



UnderSampling

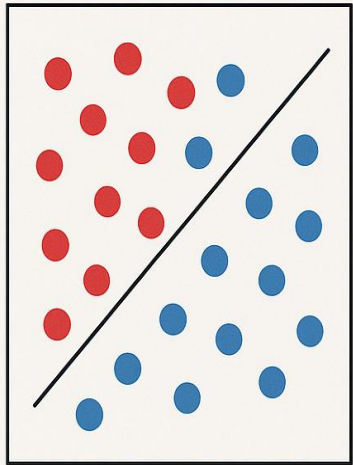


3. Model Selection & Hyperparameter Tuning

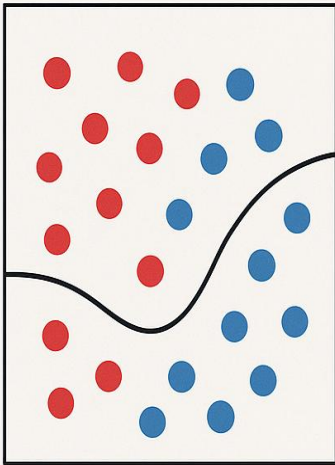
Techniques

- Choose suitable algorithms:
 - Logistic Regression
 - SVM
 - Random Forest
 - Neural Networks
- **Hyperparameter Tuning:**
 - Grid Search
 - Random Search
 - Bayesian Optimization
- **Cross-Validation:** Reduces overfitting and improves generalization.

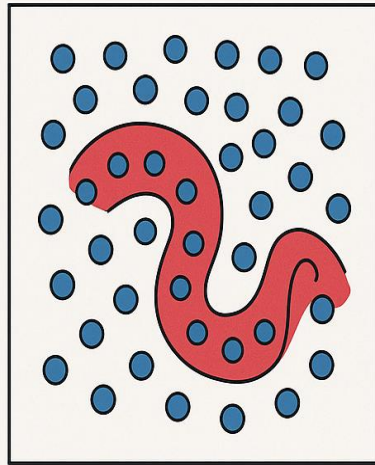
What Do Decision Boundaries Tell Us About a Model?



Linear
Classifier

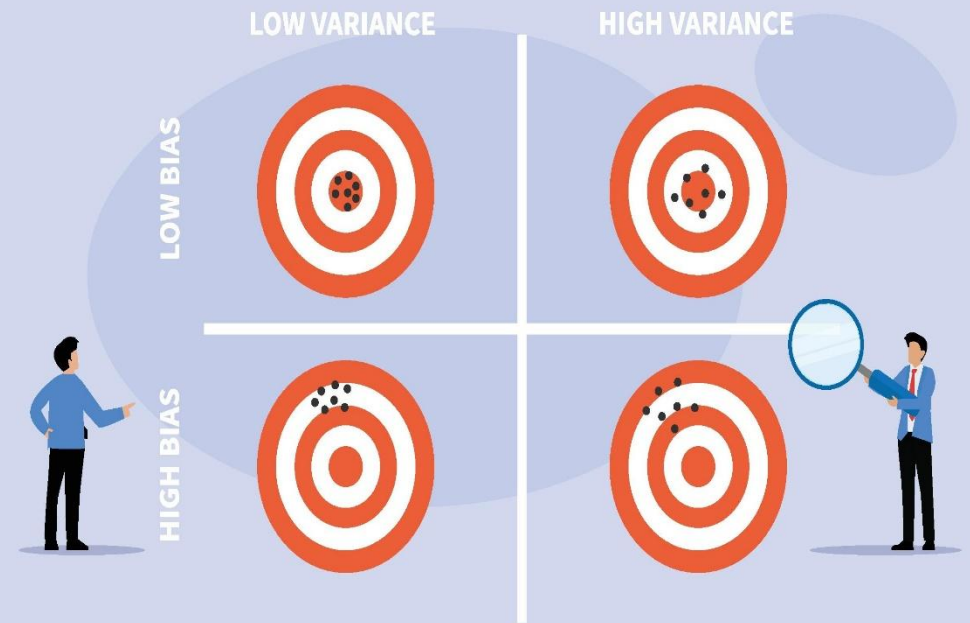


SVM



Neural
Network

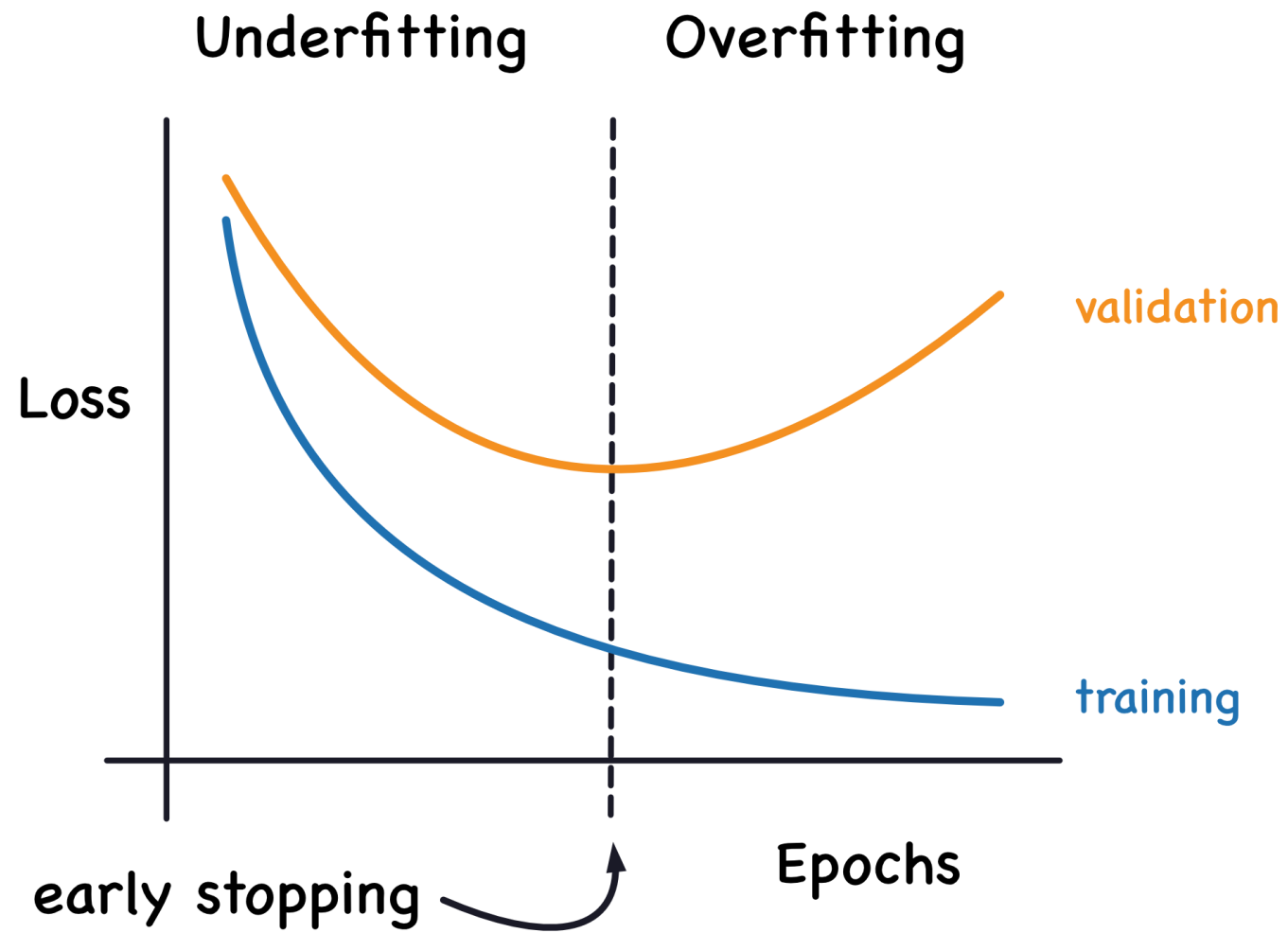
BIAS-VARIANCE TRADEOFF



4. Reducing Overfitting

Methods

- **Regularization:** L1 / L2 penalty
- **Dropout (Deep Learning):** Randomly deactivate neurons.
- **Early Stopping:** Stop training when validation loss increases.
- **Data Augmentation:** Generate more training samples.



Training (Training Loss)

- **Training loss** means the error calculated on the **training dataset**.
- It shows **how well the model is learning from the data it has seen**.
- In the graph, the **blue curve is continuously decreasing**.
- As training increases (more epochs), the model **fits the training data better**.
- Lower training loss → Model is memorizing or learning patterns.
- If training loss becomes **very low**, sometimes it indicates **overfitting**.

Validation (Validation Loss)

- **Validation loss** is the error calculated on **unseen data (validation dataset)**.
- It measures the **generalization ability** of the model.
- Initially validation loss **decreases** → Model is learning useful patterns.
- After some point, validation loss **starts increasing** → Model is not generalizing well.
- Increasing validation loss indicates **overfitting starts**.

Epochs / Training Time (X-Axis)

- Represents the **number of iterations or passes through the entire training dataset.**
- More epochs → Model learns more details.
- Too few epochs → **Underfitting**
- Too many epochs → **Overfitting**

Loss / Error (Y-Axis)

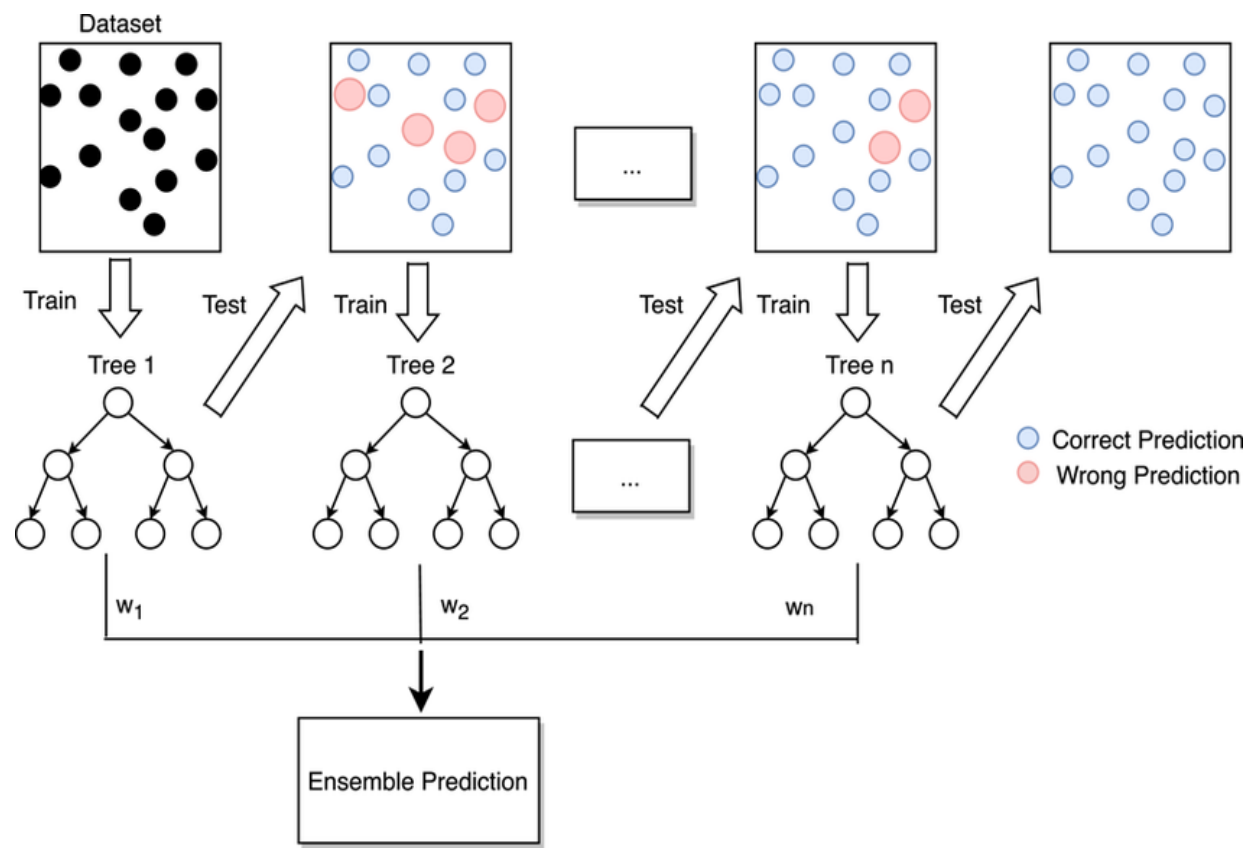
- Represents the **amount of prediction error**.
- Lower value means **better model performance**.
- **Optimal Point (Dashed Vertical Line)**
- This is the **best stopping point of training**.
- At this point:
 - Validation loss is **minimum**
 - Model has **best generalization**
- This technique is called **Early Stopping**.

- **Overfitting Region (Right Side of Dashed Line)**
- Training loss continues to decrease.
- Validation loss increases.
- Model is **memorizing training data**
- Performs **poorly on new/unseen data**
- ***“Overfitting occurs when training loss decreases continuously but validation loss starts increasing after a certain number of epochs.”***

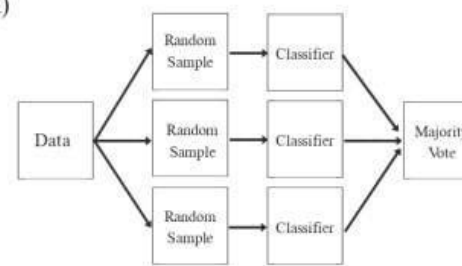
5. Ensemble Methods

Popular Approaches

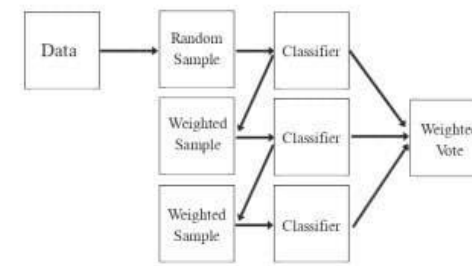
- **Bagging:** e.g., Random Forest
- **Boosting:** AdaBoost, Gradient Boosting, XGBoost
- **Stacking / Voting:** Combine multiple models.
- Ensemble models usually give **higher accuracy than single models.**



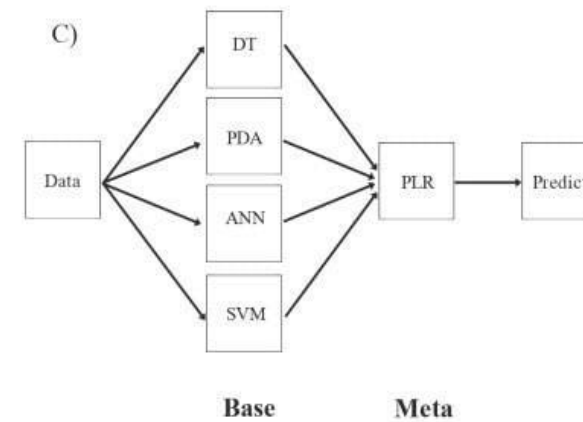
A)



B)

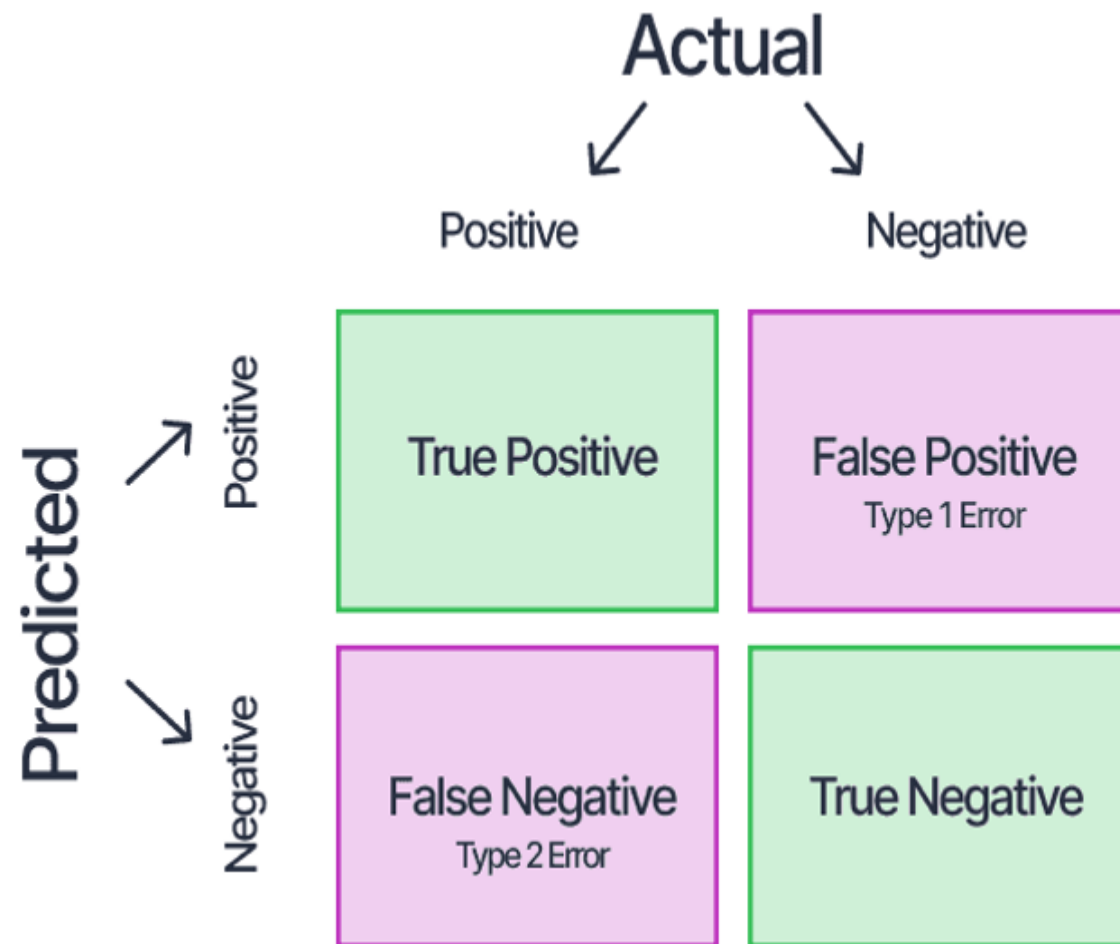
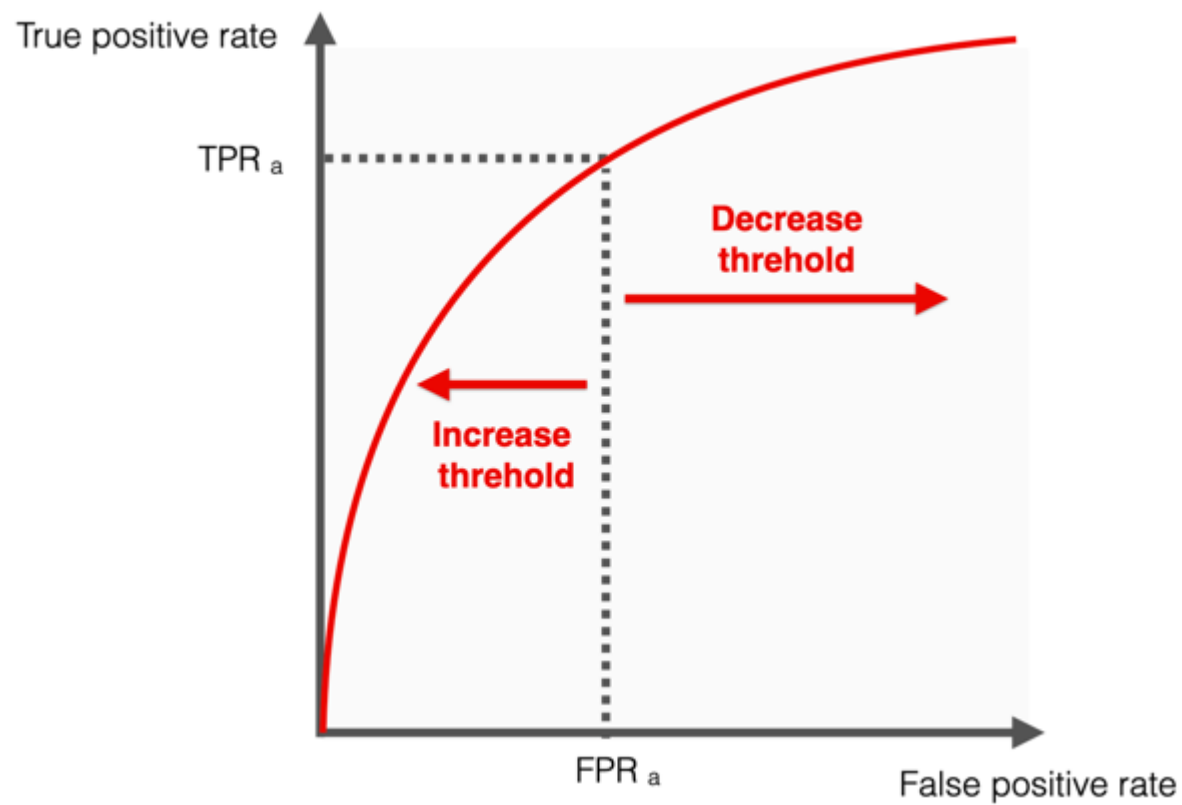


C)



6. Evaluation Metrics (Beyond Accuracy)

- Precision
- Recall
- F1-Score
- ROC-AUC
- Accuracy alone may be misleading (especially in imbalanced data).



- Improve data quality and perform feature engineering
- Handle class imbalance
- Select proper model and tune hyperparameters
- Reduce overfitting using regularization / dropout
- Use ensemble learning techniques
- Evaluate with proper performance metrics

Regression

- **Regression** is a type of **supervised machine learning** technique used to **predict a continuous output value** based on one or more input (independent) variables.
- **Regression Equation:**

$$Y=f(X)+\varepsilon$$

- Where:
- Y: Dependent variable (target/output)
- X: Independent variable(s) (input)
- $f(X)$: Function that models the relationship (e.g., linear)
- ε : Error term (difference between prediction and actual)

Predict a student's **exam score (Y)** based on the **number of hours studied (X)**.

Hours Studied (X)	Marks Scored (Y)
1	52
2	56
3	61
4	66
5	71

Apply Regression Model:

- $Y = mX + c + \epsilon$
- Where:
- X: Hours studied
- Y: Marks scored
- m Slope of the line (rate of increase in marks per hour)
- c: Intercept (marks when $X = 0$)

In regression analysis, the **intercept** is the value of the **dependent variable (Y)** when **all independent variables (X)** are zero.

- ϵ : Error term (difference between actual and predicted)

$$Y=5X+47$$

Prediction Example:

If a student studies for **6 hours**, predict the marks:

$$Y=5(6)+47=30+47=77$$

To calculate the m

- Step 1: Calculate Mean of X and Y

$$\bar{X} = \frac{1 + 2 + 3 + 4 + 5}{5} = \frac{15}{5} = 3$$

$$\bar{Y} = \frac{52 + 56 + 61 + 66 + 71}{5} = \frac{306}{5} = 61.2$$

- Step 2: Calculate the Slope (m)

$$m = \frac{\sum (X - \bar{X})(Y - \bar{Y})}{\sum (X - \bar{X})^2}$$

X	Y	X - $\bar{X}$	Y - $\bar{Y}$	(X - $\bar{X}$)(Y - $\bar{Y}$)	(X - $\bar{X}$) ²
1	52	-2	-9.2	18.4	4
2	56	-1	-5.2	5.2	1
3	61	0	-0.2	0	0
4	66	1	4.8	4.8	1
5	71	2	9.8	19.6	4

$$m = \frac{18.4 + 5.2 + 0 + 4.8 + 19.6}{4 + 1 + 0 + 1 + 4} = \frac{48}{10} = 4.8$$

- Step 3: Calculate Intercept (c)

$$c = \bar{Y} - m \cdot \bar{X}$$

$$c = 61.2 - (4.8)(3) = 61.2 - 14.4 = 46.8$$

- Final Regression Line:
- $Y = 4.8X + 46.8$

1. Linear Regression

- Linear Regression is used to model the relationship between a **single independent variable (X)** and a **dependent variable (Y)** by fitting a straight line ($Y = mX + c$) to the data.

2. Multiple Linear Regression

- Used when there are **two or more independent variables** predicting a dependent variable.

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

Predict the price of a house based on **size (sqft)** and **number of bedrooms**.

Size (X_1)	Bedrooms (X_2)	Price (Y)
1000	2	300,000
1500	3	400,000
2000	3	500,000
2500	4	600,000

- To calculate the predicted price Y using the multiple linear regression equation:

$$Y = b_0 + b_1(1800) + b_2(3)$$

- We need the actual **coefficients** b_0 , b_1 , and b_2 .

Coefficient	Value
b_0	20,000 (intercept)
b_1	200 (per sqft)
b_2	15,000 (per bedroom)

$$Y = 20,000 + 200 \times 1800 + 15,000 \times 3$$

$$Y = 20,000 + 360,000 + 45,000$$

$$Y = 425,000$$

$Y = ₹425,000$

Logistic Regression

- Used when the **dependent variable is categorical**, typically binary (0 or 1).

Instead of predicting the value directly, it predicts the **probability** that Y belongs to a certain class.

$$P(Y = 1) = \frac{1}{1 + e^{-(b_0 + b_1 X)}}$$

Predict whether a student passes (1) or fails (0) based on hours studied.

Hours Studied (X)

1

2

3

4

5

Pass (Y)

0

0

1

1

1

1. Assume the **logistic function**:

$$\text{Logit}(p) = \log \left(\frac{p}{1-p} \right) = b_0 + b_1 X$$

2. Fit the model using **maximum likelihood estimation (MLE)**.

3. Get the equation:

$$P(\text{pass}) = \frac{1}{1 + e^{-(b_0 + b_1 X)}}$$

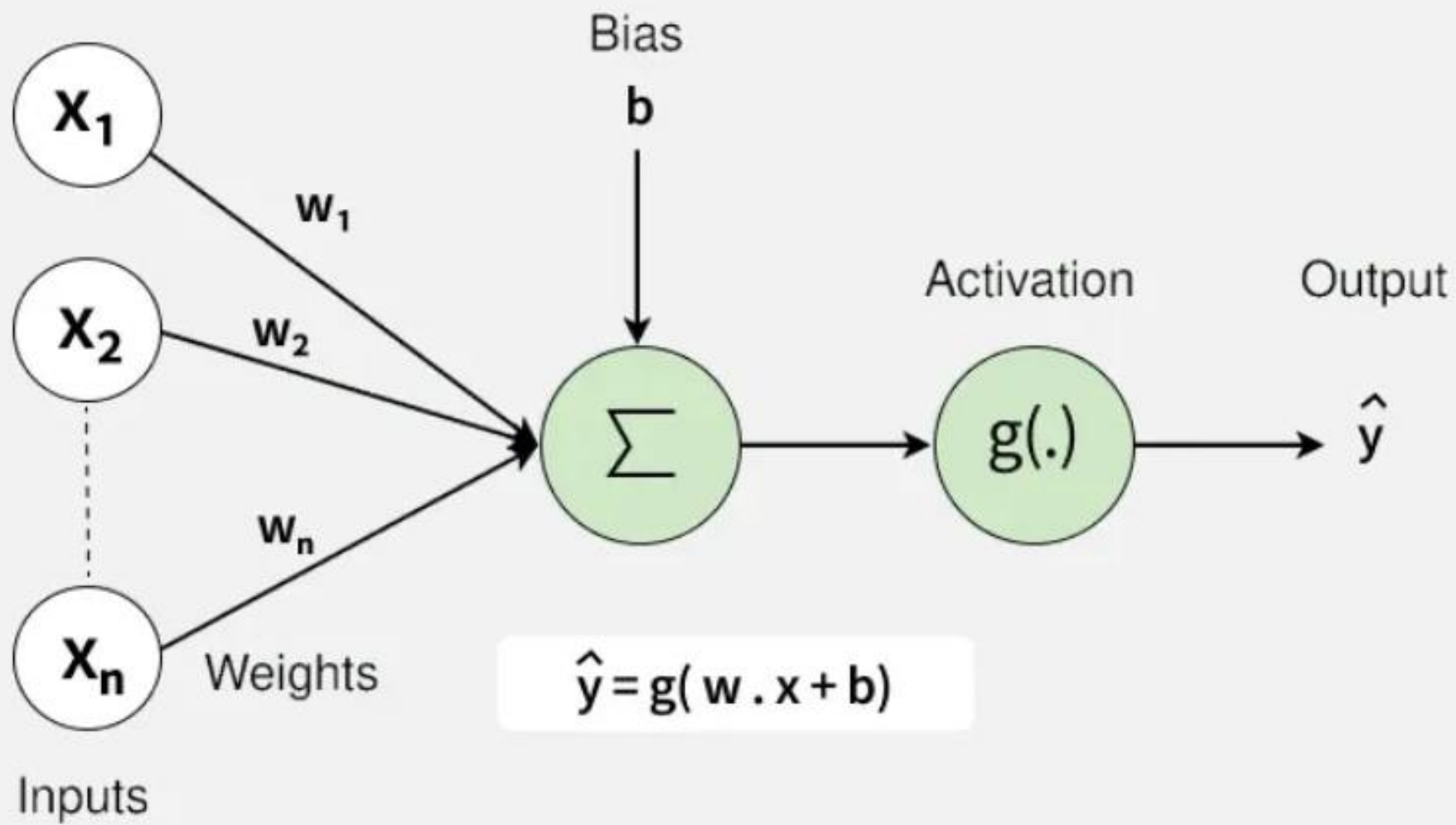
4. **Predict:**

- For a student who studied 3.5 hours, calculate the probability.
- If $P > 0.5$, classify as "Pass"; else "Fail".

Type	Dependent Variable	Independent Variables	Output
Linear Regression	Continuous	1	Continuous value
Multiple Linear Reg.	Continuous	2 or more	Continuous value
Logistic Regression	Categorical (0/1)	1 or more	Probability (0–1)

What is a Neural Network?

- Neural networks are **machine learning models** that mimic the complex functions of the human brain.
- These models consist of *interconnected nodes or neurons that process data, learn patterns* and enable tasks such as *pattern recognition and decision-making*.



Symbol	Meaning
$x_1, x_2, \dots, x_n$	Input features
$w_1, w_2, \dots, w_n$	Weights for each input
b	Bias
$\sum$	Weighted sum: $\mathbf{w} \cdot \mathbf{x} + b$
$g(z)$	Activation function
$\hat{y}$	Output prediction

- This diagram represents a **single neuron (perceptron)** in a **neural network**, showing how input data is processed to produce an output prediction.

$$\hat{y} = g(\mathbf{w} \cdot \mathbf{x} + b)$$

- This is the **forward propagation** formula for a single neuron.

1. Inputs ($x_1, x_2, \dots, x_n$)

- Represent the **features** or data given to the model.
- Example: In image recognition, each pixel value is an input.
- In a student's pass/fail prediction, x_1 might be "hours studied", x_2 could be "attendance", etc.

2. Weights ($w_1, w_2, \dots, w_n$)

- Each input is multiplied by a corresponding **weight**.
- Weights determine **how important** each input is.
- During training, the neural network **learns optimal weights** to make accurate predictions.

3. Weighted Sum (Σ)

- The neuron **calculates the dot product** of weights and inputs:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n \Rightarrow \mathbf{w} \cdot \mathbf{x}$$

- This result is called the **linear combination**.

4. Bias (b)

- A **bias term** is added to the weighted sum to **shift** the activation function.
- Helps the model learn even when all input values are 0.
- Makes the model **more flexible**.

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

Bias is a trainable constant added to the weighted sum of inputs **before** applying the activation function.

5. Activation Function (g(.))

- Applies a **non-linear transformation** to the input.
- Most commonly used activation in classification tasks: **Sigmoid function**:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Rectified Linear Unit

Hyperbolic Tangent

- Other activations: ReLU, Tanh, etc.
- Without this, the network would behave like a linear model.

6. Output ($\hat{y}$ or y-hat)

- This is the **final output** from the neuron.
- For binary classification (like pass/fail), it's a probability:

$$\hat{y} = g(\mathbf{w} \cdot \mathbf{x} + b)$$

- Value ranges between **0 and 1** (if using sigmoid), representing the **probability of class 1**.